

Lab 11 — Proving It Works: Unit Testing with Unity

ESP32-S3 FIRMWARE LAB SERIES · PHASE 2 · PAST THE COURSEWORK

Estimated time	2 hrs
Introduces	testability by extraction, known-answer tests, float tolerance
Prerequisites	Lab 7 (the feature functions under test).
Beyond the board	Nothing. The interactive test menu needs the UART-bridge USB port.
Target	ESP32-S3 (written against an N16R8 module: 16 MB flash, 8 MB octal PSRAM)
Framework	ESP-IDF v5.5, CMake and Ninja, driven by <code>idf.py</code>

NOTE

How to read this lab

Sections run **§11.1 objective**, **§11.2 parts and wiring**, **§11.3 how it works**, then the code, a line-by-line walkthrough, the documentation map, and verification. Callout boxes are tagged **[WARNING]**, **[KEY POINT]**, **[NOTE]**, **[DOCS]** and **[TM4C BRIDGE]**. Figures are given as captioned text and tables so every value in them is searchable and checkable.

The **TM4C bridge** notes map each topic onto the equivalent bare-metal, register-level work on a TI TM4C123 (Cortex-M4F). Skip them if that is not your background; nothing later depends on them.

11.1 Objective & TM4C bridge

Prove your code is correct automatically instead of by squinting at the monitor. This lab is less about a peripheral and more about a habit that visibly separates junior from senior firmware engineers.

TM4C BRIDGE

Nothing like this existed in your bare-metal coursework. You validated by watching an LED or a scope, which tells you the system works *right now, on this board, for the case you happened to try*. Automated tests tell you it still works after every future change, which is a completely different guarantee.

11.2 Parts & wiring

Item	Status
ESP32-S3 board	nothing external

No hardware, deliberately. The whole point is that the logic under test has been separated *from* the hardware, so the tests need no sensor attached. That separation is the lab.

11.3 How it works

This lab has no peripheral. It is about a habit, and the habit is worth more than most peripherals.

How you have been testing so far, and why it does not scale

Every lab up to now was verified the same way: flash it, watch the monitor or the LED, decide it looks right. That is **manual verification**, and it has three specific weaknesses.

It only tests what you happened to try. You shook the board and RMS went up. Did you test a window of all zeros? A single enormous spike? Exactly one sample? Almost certainly not.

It does not stay tested. You verified Lab 7's maths in August. In October you optimise the loop. Is it still correct? The only way to know is to redo the whole manual check, so in practice nobody does, and bugs get introduced silently.

It needs the hardware. You cannot check the feature maths without a board, a wired sensor and something to shake.

Automated tests fix all three: they cover the cases you choose deliberately, they re-run in seconds on every change, and — if you structure the code right — they need no hardware at all.

The refactor that makes testing possible

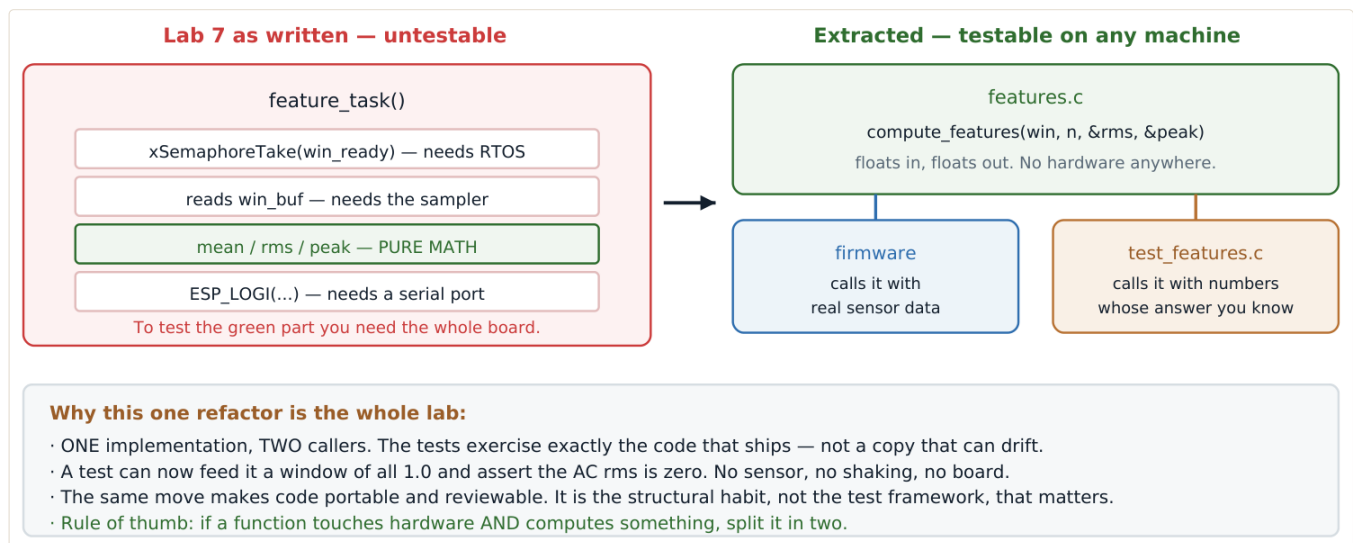


Figure 11.1. The move at the heart of this lab. Pull the pure computation out of the task that talks to hardware, and it becomes callable by both the firmware and a test. One implementation, two callers — so the tests exercise exactly the code that ships.

Tangled code versus logic extracted for testing.

Left, Lab 7 as written, untestable. `feature_task()` contains four things welded together: 1. `xSemaphoreTake(win_ready)` -- needs the RTOS 2. reads `win_buf` -- needs the sampler task 3. `mean / rms / peak` -- PURE MATH, the only part you actually want to test 4. `ESP_LOGI(...)` -- needs a serial port

To test the pure-math part you would need the whole board.

Right, extracted and testable on any machine. `features.c` exposes `compute_features(win, n, &rms, &peak)`: floats in, floats out, no hardware anywhere. Two callers now share it: the firmware calls it with real sensor data, and `test_features.c` calls it with numbers whose answer you already know.

Why this one refactor is the whole lab: - ONE implementation, TWO callers. The tests exercise exactly the code that ships, not a copy that can drift. - A test can feed it a window of all 1.0 and assert the AC rms is zero. No sensor, no shaking, no board. - The same move makes code portable and reviewable. - Rule of thumb: if a function touches hardware AND computes something, split it in two.

Look at Lab 7's `feature_task` honestly. It blocks on a semaphore, reads a buffer another task fills, computes RMS and peak, and logs the result. Only the third of those four things is *logic*; the rest is plumbing. And because they are welded together in one function, you cannot reach the logic without the plumbing.

The fix is to lift the computation into a function that takes plain data and returns plain results, touching nothing else:

```
void compute_features(const float *win, int n, float *rms_out, float *peak_out);
```

Now the firmware calls it with real sensor data, and a test calls it with numbers whose correct answer is already known. **Same function, both times.** That last point is what makes the tests meaningful — a test that exercises a *copy* of the logic proves nothing about the code you actually ship.

The rule of thumb generalises well beyond this lab: **if a function both touches hardware and computes something, split it in two.** The half that computes becomes testable, portable and reviewable. The half that touches hardware stays small enough to verify by inspection.

What makes a test worth writing

A test that merely runs the code and checks it did not crash is nearly worthless. The strongest kind is a **known-answer test**: an input whose correct output you can derive by hand, independently of the implementation.

Input	Correct answer	Why you know it without running the code
64 samples, all exactly 1.0	rms = 0, peak = 0	Every sample equals the mean, so every deviation is zero.
{1, -1, 1, -1}	rms = 1.0	Mean is 0; every deviation has magnitude 1; the mean of 1 ² is 1; $\sqrt{1} = 1$.
{0, 0, 0.5, 0}	peak > 0.3	One sample sits well away from a mean of 0.125.

Notice you can verify every one of those on paper. That independence is exactly the point: if the test agreed with the code because it was derived *from* the code, it would pass even when both were wrong.

Beyond known answers, test the **edges**, because bugs cluster at boundaries: a window of length 1, a window of all zeros, the largest and smallest values the type can hold, and — the one this code actually gets wrong — a length of 0, which divides by zero.

Why you never compare floats with ==

This catches everyone once and it is a standard interview question.

Floating-point numbers are stored as a sign, an exponent and a fixed number of significand bits. Most decimal values cannot be represented exactly in that format, in the same way one third cannot be written exactly in decimal. So arithmetic **rounds**, and rounding errors accumulate across a calculation.

The practical consequence: a computation whose exact mathematical answer is 0 may produce `0.00000007`. That result is *correct* — it is as close to zero as this many bits allows — but `result == 0.0f` is false, and a test written that way fails on perfectly good code.

So you assert that a value lies **within a tolerance** of the expected one:

```
TEST_ASSERT_FLOAT_WITHIN(1e-4, 0.0f, rms); // passes if |rms - 0.0| < 0.0001
```

Choosing the tolerance is a judgement call: loose enough to absorb rounding, tight enough that a real error still fails. For accumulated sums over 256 samples, `1e-4` is comfortable.

What Unity is

Unity is a small C test framework that ships with ESP-IDF. It is deliberately minimal — a set of assertion macros, a way to register test cases, and a runner — because it has to work on microcontrollers with no filesystem and no dynamic allocation.

You declare a test with `TEST_CASE("name", "[tag]")`, and it registers itself automatically; there is no list to maintain. The runner then prints a menu over serial and lets you run one test or all of them. Nothing about it is ESP-specific — Unity is widely used across embedded, so this transfers.

NOTE

Where this goes in a real project

On a professional team these tests do not get run by a human pressing a key. They run automatically on every commit, on real hardware, through a CI system — ESP-IDF's `pytest-embedded` exists for exactly that. You are building the piece that makes that possible. Saying "I extracted the signal-processing logic so it could be unit-tested independently of the hardware" is a genuinely senior-sounding sentence, and after this lab it is true.

11.4 Procedure & code

Three pieces: the extracted logic, the tests, and a test runner that replaces your normal `app_main` for this build.

```
// ---- features.h : the declaration both the firmware and the tests include ----
void compute_features(const float *win, int n, float *rms_out, float *peak_out);

// ---- features.c : pure logic, no hardware anywhere in sight ----
#include <math.h>
#include "features.h"

void compute_features(const float *win, int n, float *rms_out, float *peak_out) {
```

```

float mean = 0;
for (int i = 0; i < n; i++) mean += win[i];
mean /= n;
float ss = 0, peak = 0;
for (int i = 0; i < n; i++) {
    float d = win[i] - mean;
    ss += d * d;
    if (fabsf(d) > peak) peak = fabsf(d);
}
*rms_out = sqrtf(ss / n);
*peak_out = peak;
}

```

```

// ---- test/test_features.c ----
#include "unity.h"
#include "features.h"

TEST_CASE("DC signal has zero AC rms", "[features]") {
    float w[64];
    for (int i = 0; i < 64; i++) w[i] = 1.0f;    // constant: no variation at all
    float rms, peak;
    compute_features(w, 64, &rms, &peak);
    TEST_ASSERT_FLOAT_WITHIN(1e-4, 0.0f, rms);    // within tolerance, never ==
    TEST_ASSERT_FLOAT_WITHIN(1e-4, 0.0f, peak);
}

TEST_CASE("a single spike is caught by peak", "[features]") {
    float w[4] = { 0, 0, 0.5f, 0 };
    float rms, peak;
    compute_features(w, 4, &rms, &peak);
    TEST_ASSERT_TRUE(peak > 0.3f);
}

TEST_CASE("alternating +/-1 has rms of 1", "[features]") {
    float w[4] = { 1, -1, 1, -1 };    // mean 0, every deviation magnitude 1
    float rms, peak;
    compute_features(w, 4, &rms, &peak);
    TEST_ASSERT_FLOAT_WITHIN(1e-4, 1.0f, rms);
}

```

```

// ---- the test app's entry point ----
#include "unity.h"
void app_main(void) {
    unity_run_menu();    // lists the tests over serial; you pick which to run
}

```

11.5 Code walkthrough, line by line

Three files rather than one, and the separation between them is the lesson.

Block 1, line 2 · `void compute_features(const float win, int n, float rms_out, float *peak_out);`

The declaration, and every part of the signature is a decision.

`const float *win` is a pointer to floats the function promises **not** to modify. Read it right to left: pointer, to `float`, which is `const`. This is a contract enforced by the compiler — try to assign through it and the build fails. It also documents intent to the caller: hand me your buffer, I will not disturb it.

`int n` is the element count, passed separately because **C arrays do not carry their length**. Inside the function `win` is just an address; `sizeof(win)` would give you the size of a pointer, not the array. Passing the count alongside the pointer is universal in C and is the source of a whole family of buffer bugs when the two disagree.

`float *rms_out`, `float *peak_out` are **output parameters**. A C function can `return` only one value, and this one has two answers, so the extras come back through pointers the caller supplies. You have already used this mechanism without naming it: `&last` in Lab 2, `&strip` in Lab 1, `&h` in Lab 10.

Block 1, lines 5-6 · `#include / #include "features.h"`

`math.h` for `sqrtf` and `fabsf`. Including its own header, `features.h`, is a deliberate habit: it makes the compiler check the definition against the declaration, so if they ever drift apart you get an error instead of a link-time surprise.

Block 1, lines 8-11 · the mean, first pass

Accumulate every sample, divide by `n`. Note this uses the parameter `n` rather than a hard-coded 256 — which is exactly what lets a test call it with a 4-element array.

Block 1, lines 12-17 · the deviations, second pass

`float d = win[i] - mean;` is one sample's deviation. `ss += d * d;` accumulates squares so negative and positive count alike. `if (fabsf(d) > peak) peak = fabsf(d);` tracks the largest absolute deviation.

Why two passes. Every deviation is measured from the mean, and the mean is not known until every sample has been seen, so one pass is impossible with this formula. A one-pass alternative exists using the sum and the sum of squares, and it is faster — but it subtracts two large nearly-equal numbers, which loses precision badly when the mean dominates the variation. That is exactly this case, since gravity sits at 1.0 g and the vibration you care about may be 0.01. The two-pass version is the numerically stable choice, and that is the answer if an interviewer asks why you did it the slow way.

Block 1, lines 18-19 · `rms_out = sqrtf(ss / n); peak_out = peak;`

The **dereference** operator `*` writes *through* the pointer into the caller's variable. `rms_out` is an address; `*rms_out` is the float living at that address. Forgetting the star here is a classic error — `rms_out = ...` would assign to the local copy of the pointer and silently do nothing useful.

WARNING

The bug this function still has

If `n` is 0, line 11 divides by zero. On this chip that produces `NaN` rather than a crash, which then propagates silently through every later calculation — and `NaN` compares false against everything, including itself. §11.7 asks you to write the test that catches this *before* fixing it, which is the red-green cycle in miniature.

Block 2, line 2 · `#include "unity.h"`

The test framework. Everything below — `TEST_CASE`, the assertions — comes from here.

Block 2, line 5 · `TEST_CASE("DC signal has zero AC rms", "[features])`

Declares one test. Two arguments: a **human-readable name** that appears in the runner menu and in failure output, and a bracketed **tag** for grouping. Tags let you run all `[features]` tests at once and skip slower groups.

The macro registers the test automatically at build time using linker section tricks, which is why there is no list of tests to maintain anywhere. Adding a test is just writing one.

Name tests by **what they assert**, not by what they call. "DC signal has zero AC rms" tells you what broke when it fails; "test_compute_features_1" does not.

Block 2, lines 6-7 · `float w[64]; for (...) w[i] = 1.0f;`

Builds the input: 64 identical samples. A constant signal has no variation at all, so its AC RMS must be exactly zero — derivable on paper, independent of the implementation. That is what makes it a known-answer test.

Block 2, line 9 · `compute_features(w, 64, &rms, &peak);`

Calls the real function — the same one the firmware calls — passing addresses so the results come back. No board, no sensor, no RTOS.

Block 2, lines 10-11 · `TEST_ASSERT_FLOAT_WITHIN(1e-4, 0.0f, rms);`

Argument order is **tolerance, expected, actual**, and it is easy to get backwards. It passes if `|actual - expected| < tolerance`.

`1e-4` is scientific notation for 0.0001. Never `==` here, for the reasons in §11.3.

Block 2, lines 14-20 · the spike test

`{0, 0, 0.5f, 0}` has a mean of 0.125, so the spike deviates by 0.375 and the flat samples by 0.125. The assertion `peak > 0.3f` confirms peak-tracking found the spike rather than the baseline.

Note this uses `TEST_ASSERT_TRUE` with an inequality rather than asserting an exact value. That is a deliberate choice: the test captures the *behaviour* you care about — "the spike dominates" — without over-specifying an exact number that would break if you legitimately changed the definition. Tests that assert more than they need to are brittle.

Block 2, lines 22-28 · the alternating test

`{1, -1, 1, -1}` has a mean of 0 and every deviation has magnitude 1, so the mean square is 1 and the RMS is exactly 1. A second independent known answer, and one that would catch a missing square root that the first test would not — because with an answer of zero, `sqrt(0)` and `0` look identical. **Two tests that fail differently are worth far more than two that fail the same way.**

Block 3 · `void app_main(void) { unity_run_menu(); }`

The test build's entry point. `unity_run_menu()` lists every registered `TEST_CASE` over serial and waits for you to pick one, or run them all.

Notice what this implies structurally: the test app has its **own app_main**, replacing the firmware's. The firmware and the tests are **two different programs that share one features.c**. That is the entire

payoff of the extraction, made concrete — and it is why the logic had to come out of the task in the first place.

11.6 Docs & references

DOCS

Trace every line to its source

Unity and the ESP-IDF test framework — API Guides → **Unit Testing in ESP-IDF**. This explains the `test/` directory layout, how test components are built, and how to run them.

Unity assertion reference — the full macro list ships with ESP-IDF under `components/unity/`. Read the header; it is the fastest way to find the right assertion.

`const` correctness — any good C reference. Worth understanding properly, since interviewers ask what `const float *p` means versus `float * const p`.

pytest-embedded — mentioned in the same testing guide. This is how tests get run automatically in CI on real hardware, which is what a professional setup looks like.

11.7 Verify, troubleshoot, your turn

Verify. Flash the test app and the monitor shows a menu of your tests. Run them all; each should report PASS and you get a summary count. Then **deliberately break the code** — change `sqrtf(ss / n)` to `ss / n` — and confirm the tests fail with a useful message. A test suite you have never seen fail is a test suite you should not trust.

Troubleshoot

Symptom	Cause and fix
No tests found	The test file is not in a directory the build recognises as tests. Check the layout against the guide.
Link error on <code>compute_features</code>	The test component does not list the component under test in its requirements.
Float test fails by a tiny amount	Your tolerance is too tight, or you used <code>==</code> .
Tests pass but the real firmware misbehaves	The tests are correct and incomplete. Add a case covering whatever the firmware actually hit.

Your turn

1. **Write a failing test first.** Pick a behaviour `compute_features` does not have yet — for instance, returning something sensible when `n` is 0, which currently divides by zero. Watch it fail, then fix the function, then watch it pass. That red-green cycle is test-driven development in miniature.
2. **Test the byte reassembly from Lab 5.** Extract the `(int16_t)((hi << 8) | lo)` conversion into its own function and write known-answer tests: `0x4000` must give +16384, `0xC000` must give −16384. This proves the signed cast is doing its job and would have caught the classic bug immediately.

3. **Refactor Lab 7 to use `features.c`** so the firmware and tests genuinely share one implementation. Now a change to the maths is validated automatically before it ever reaches the board.

Bench realities

Problems that stop this lab but are not about the peripheral it teaches. Each one cost real time on a real bench.

The Unity test menu is empty

WARNING

Symptom: Lab 11 builds and runs, the menu prompt appears, and it lists no tests at all.

Cause: `TEST_CASE` registers each test by placing a record in a dedicated linker section. But your test file defines no symbol that anything else calls, so the linker considers the whole object file unreferenced and **discards it** — taking the registrations with it.

Fix — in `main/CMakeLists.txt`:

```
idf_component_register(SRCS "main.c" "features.c" "test_features.c"
                      INCLUDE_DIRS "."
                      WHOLE_ARCHIVE)
```

`WHOLE_ARCHIVE` tells the linker to keep every object file whether or not it appears to be referenced. This is a genuinely instructive bug: it is the linker doing exactly its job, and the test framework relying on a side effect the linker cannot see.

NOTE

And the watchdog that fires while you sit at the menu

With the Unity menu open and idle, the task watchdog will trip about every five seconds. That is **expected**, not a fault: `esp_unity_readline` polls the UART in a tight loop without yielding, so the idle task is starved. It stops the moment you run a test. Do not go debugging it.

Which USB port for which job

Task	Port	Why
Flashing and normal monitoring	Either	Both enumerate a serial port
Unity interactive menu	UART bridge port (COM6 on this bench)	Unity reads with a ROM-level <code>uart_rx_one_char</code> , which is bound to UART0, not the native USB device
JTAG debugging (Lab 21)	Native USB port	The USB-Serial-JTAG peripheral is only on that port

If a lab needs you to *type* at the device and nothing you type registers, you are almost certainly on the wrong port.

The series at a glance

Each lab is a complete, independently buildable ESP-IDF project. This is Lab 11.

#	Lab	Phase
00	Toolchain: hello, ESP32-S3	Foundations
01	GPIO: RGB LED and BOOT button	Foundations
02	FreeRTOS tasks and drift-free timing	Foundations
03	UART and the loopback test	Foundations
04	Hardware timers and the 1 kHz problem	Foundations
05	I ² C and the IMU bring-up	Foundations
06	PWM with the LEDC peripheral	Foundations
07	The sensor pipeline (capstone)	Foundations
08	GPIO interrupts and ISR-safe code	Past the coursework
09	SPI, full-duplex and fast	Past the coursework
10	NVS: data that survives power loss	Past the coursework
11 ←	Unit testing with Unity	Past the coursework
12	Deep sleep and waking up	Past the coursework
13	Wi-Fi station and JSON telemetry	Past the coursework
14	BLE GATT with NimBLE	Past the coursework
15	OTA and the rollback safety net	Past the coursework
16	Priority inversion and mutexes	Past the coursework
17	CAN (TWAI): broadcast by message ID	Past the coursework
18	PID, anti-windup and derivative kick	Past the coursework
19	The ML inference harness	Gap-closers
20	Continuous ADC: DMA-backed acquisition	Gap-closers
21	On-chip debugging: JTAG, OpenOCD, GDB	Gap-closers
22	A real display (SPI TFT + DMA)	Gap-closers
23	Production test mode	Gap-closers

Written against ESP-IDF v5.5 on an ESP32-S3 N16R8 module. Pin assignments are choices, not constraints — the ESP32-S3 GPIO matrix routes almost any peripheral to almost any pin, with the exception of GPIO26–GPIO37, which are committed to the on-board flash and PSRAM and must never be used.